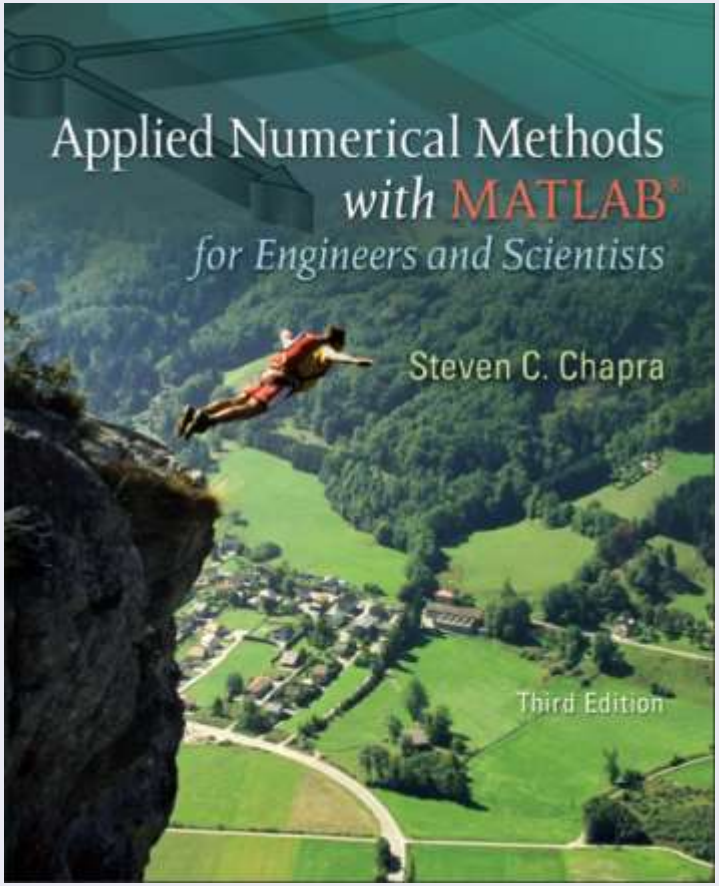


Part 2
Chapter 7

Optimization

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

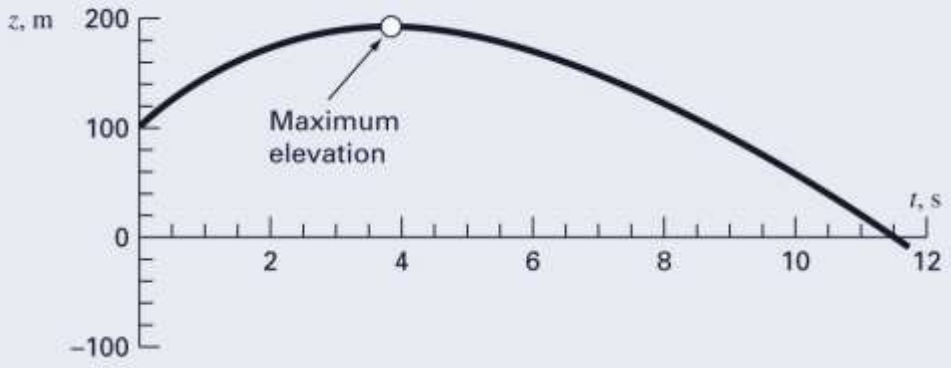
- Understanding why and where optimization occurs in engineering and scientific problem solving.
- Recognizing the difference between one-dimensional and multi-dimensional optimization.
- Distinguishing between global and local optima.
- Knowing how to recast a maximization problem so that it can be solved with a minimizing algorithm.
- Being able to define the golden ratio and understand why it makes one-dimensional optimization efficient.

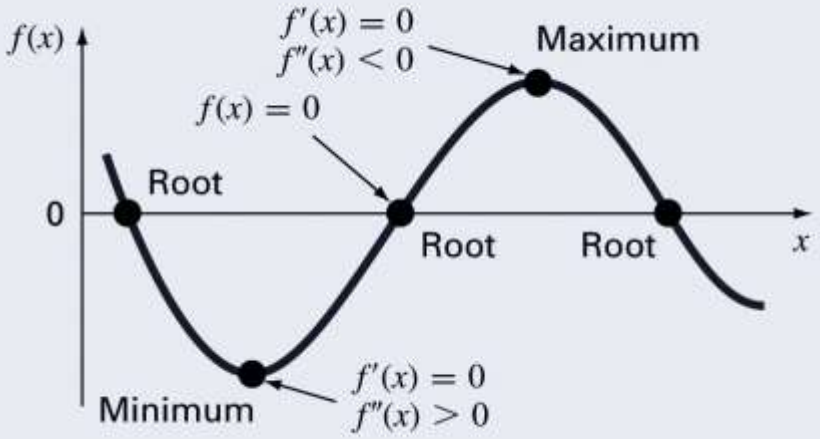
Objectives (cont)

- Locating the optimum of a single-variable function with the golden-section search.
- Locating the optimum of a single-variable function with parabolic interpolation.
- Knowing how to apply the `fminbnd` function to determine the minimum of a one-dimensional function.
- Being able to develop MATLAB contours and surface plots to visualize two-dimensional functions.
- Knowing how to apply the `fminsearch` function to determine the minimum of a multidimensional function.

Optimization

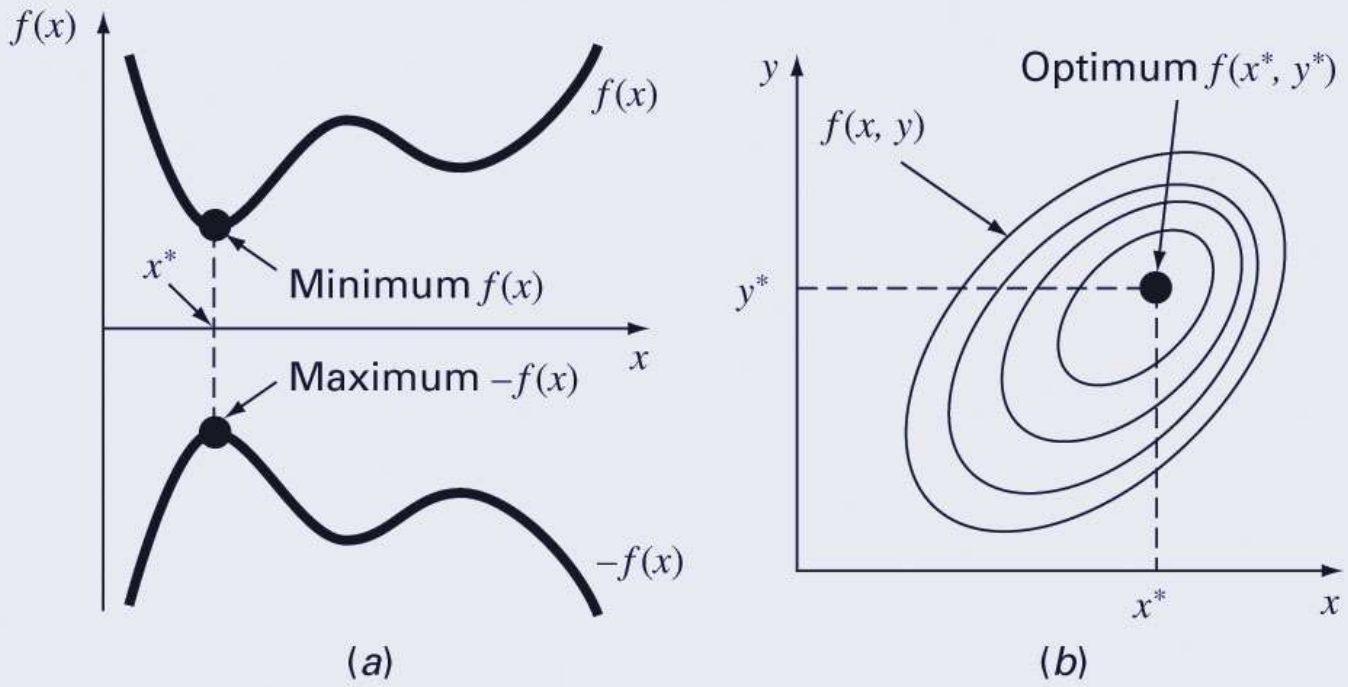
- Optimization is the process of creating something that is as effective as possible.
- From a mathematical perspective, optimization deals with finding the maxima and minima of a function that depends on one or more variables.





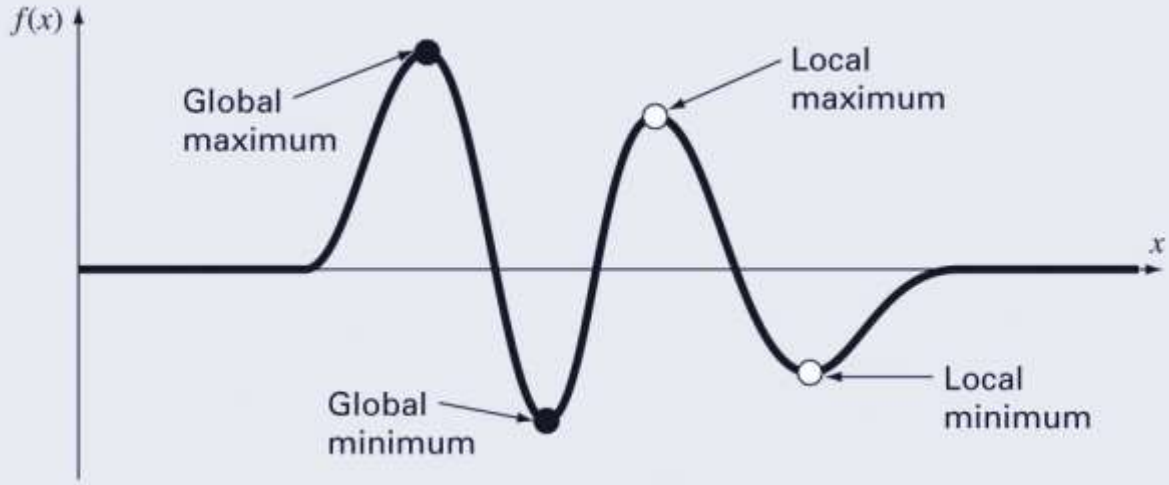
Multidimensional Optimization

- One-dimensional problems involve functions that depend on a single dependent variable -for example, *f*(*x*).
- Multidimensional problems involve functions that depend on two or more dependent variables - for example, *f*(*x*,*y*)



Global vs. Local

- A *global optimum* represents the very best solution while a *local optimum* is better than its immediate neighbors. Cases that include local optima are called *multimodal*.
- Generally desire to find the global optimum.



Golden-Section Search

- Search algorithm for finding a minimum on an interval $[x_l\ x_u]$ with a *single* minimum (*unimodal* interval)
- Uses the *golden ratio* ϕ=1.6180… to determine location of two interior points x_1 and x_2; by using the golden ratio, one of the interior points can be re-used in the next iteration.

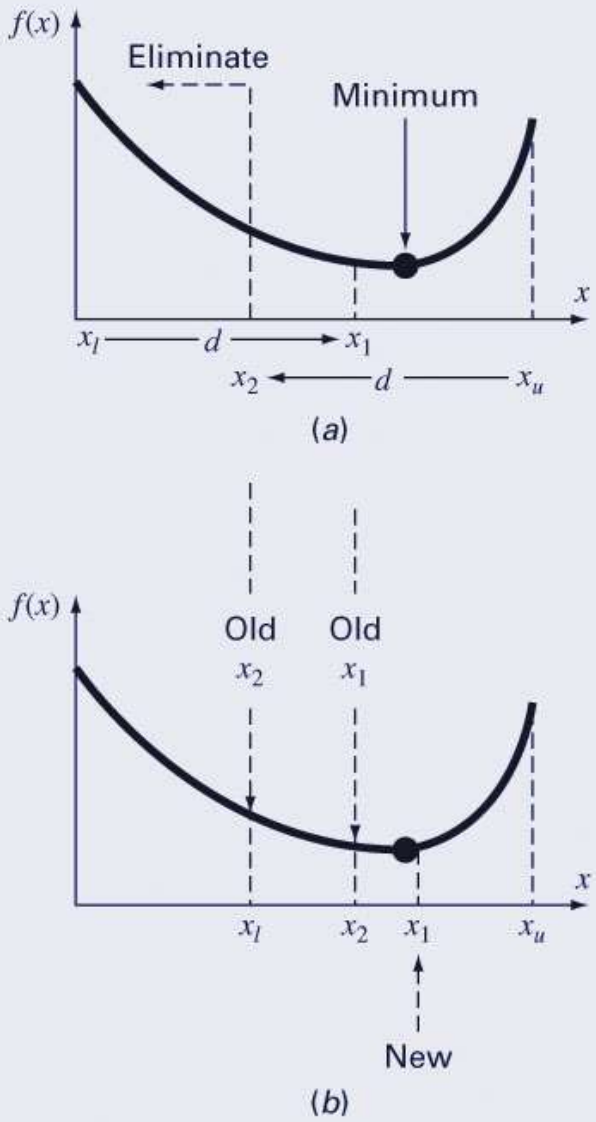
Golden-Section Search (cont)

$$d = (\phi - 1)(x_u - x_l)$$

$$x_1 = x_l + d$$

$$x_2 = x_u - d$$

- If $f(x_1) < f(x_2)$, x_2 becomes the new lower limit and x_1 becomes the new x_2 (as in figure).
- If $f(x_2) < f(x_1)$, x_1 becomes the new upper limit and x_2 becomes the new x_1.
- In either case, only one new interior point is needed and the function is only evaluated one more time.



Code for Golden-Section Search

```
function [x,fx,ea,iter]=goldmin(f,xl,xu,es,maxit,varargin)
% goldmin: minimization golden section search
%   [xopt,fopt,ea,iter]=goldmin(f,xl,xu,es,maxit,p1,p2,...):
%      uses golden section search to find the minimum of f
% input:
%   f = function handle
%   xl, xu = lower and upper guesses
%   es = desired relative error (default = 0.0001%)
%   maxit = maximum allowable iterations (default = 50)
%   p1,p2,... = additional parameters used by f
% output:
%   x = location of minimum
%   fx = minimum function value
%   ea = approximate relative error (%)
%   iter = number of iterations

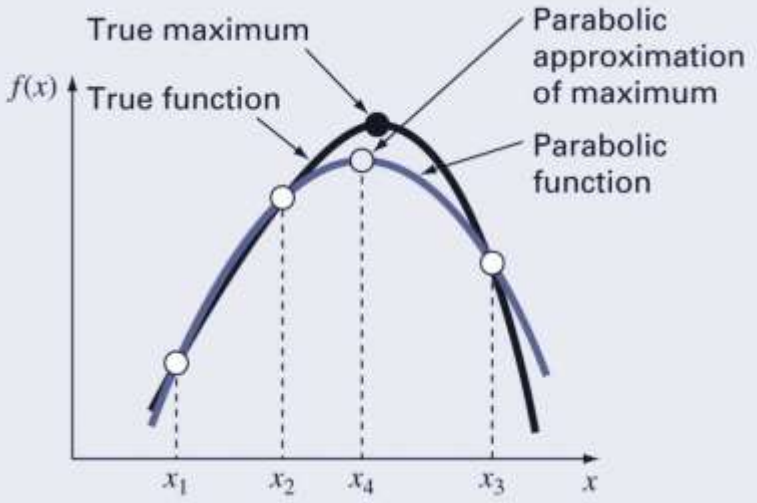
if nargin<3,error('at least 3 input arguments required'),end
if nargin<4||isempty(es), es=0.0001;end
if nargin<5||isempty(maxit), maxit=50;end
phi=(1+sqrt(5))/2;
iter=0;
while(1)
  d = (phi-1)*(xu - xl);
  x1 = xl + d;
  x2 = xu - d;
  if f(x1,varargin{:}) < f(x2,varargin{:})
    xopt = x1;
    xl = x2;
  else
    xopt = x2;
    xu = x1;
  end
  iter=iter+1;
  if xopt~=0, ea = (2 - phi) * abs((xu - xl) / xopt) * 100;end
  if ea <= es || iter >= maxit,break,end
end
x=xopt;fx=f(xopt,varargin{:});
```

Parabolic Interpolation

- Another algorithm uses parabolic interpolation of three points to estimate optimum location.
- The location of the maximum/minimum of a parabola defined as the interpolation of three points (x_1, x_2, and x_3) is:

$$x_4 = x_2 - \frac{1}{2}\frac{(x_2 - x_1)^2[f(x_2) - f(x_3)] - (x_2 - x_3)^2[f(x_2) - f(x_1)]}{(x_2 - x_1)[f(x_2) - f(x_3)] - (x_2 - x_3)[f(x_2) - f(x_1)]}$$

- The new point x_4 and the two surrounding it (either x_1 and x_2 or x_2 and x_3) are used for the next iteration of the algorithm.

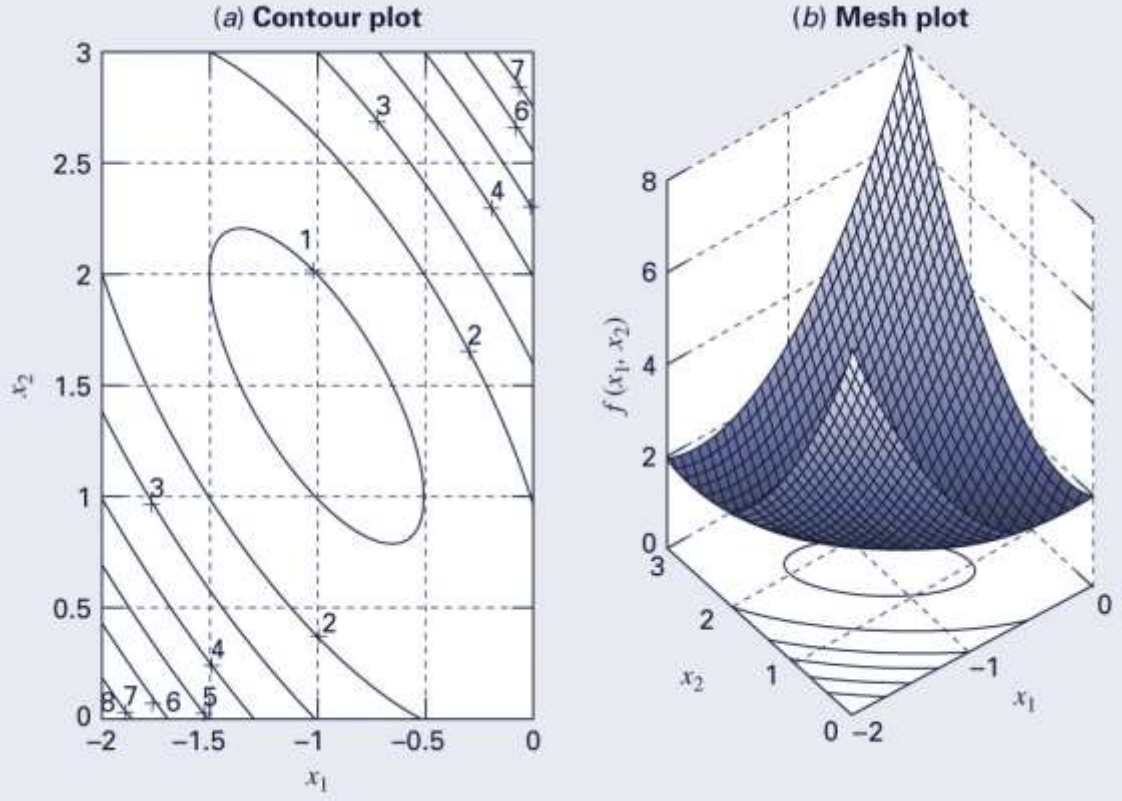


`fminbnd` Function

- MATLAB has a built-in function, `fminbnd`, which combines the golden-section search and the parabolic interpolation.
  - `[xmin, fval] = fminbnd(function, x1, x2)`
- Options may be passed through a fourth argument using `optimset`, similar to `fzero`.

Multidimensional Visualization

- Functions of two-dimensions may be visualized using contour or surface/mesh plots.



`fminsearch` Function

- MATLAB has a built-in function, `fminsearch`, that can be used to determine the minimum of a multidimensional function.
  - `[`*`xmin`*`, `*`fval`*`] = fminsearch(`*`function`*`, `*`x0`*`)`
  - *`xmin`* in this case will be a row vector containing the location of the minimum, while *`x0`* is an initial guess. Note that *`x0`* must contain as many entries as the function expects of it.
- The `function` must be written in terms of a single variable, where different dimensions are represented by different indices of that variable.

`fminsearch` Function

- To minimize
  $f(x,y)=2+x-y+2x^2+2xy+y^2$
  rewrite as
  $f(x_1, x_2)=2+x_1-x_2+2(x_1)^2+2x_1x_2+(x_2)^2$
- ```
  f=@(x) 2+x(1)-x(2)+2*x(1)^2+2*x(1)*x(2)+x(2)^2
  [x, fval] = fminsearch(f, [-0.5, 0.5])
  ```
- Note that *x0* has two entries - *f* is expecting it to contain two values.
- MATLAB reports the minimum value is 0.7500 at a location of [-1.000 1.5000]
  ```